SENIOR 3D WEB ENGINEER · INFRACLOUD3D

Mike Smahin

Senior Frontend / 3D Engineer · Barcelona

15+ years shipping frontend and real-time 3D for the web and games

I work on complex frontend systems, real-time 3D and GPU rendering. Mathematics and physics sit underneath that. I own the architecture, the performance, the asset delivery and the release.

TypeScript · Angular · React · Three.js · WebGL · WebGPU · WGSL · TSL · Unity / C#

iodiot.github.io · github.com/iodiot · @iodiot · iodiot@gmail.com

01 · WHO I AM

I make demanding real-time 3D run smoothly in the browser, and ship it as product.

I'm a Senior Software Engineer with around 15 years of professional experience, focused on complex frontend applications, real-time 3D graphics and performance-critical systems. Most recently, at RedCore, I had technical ownership of RedCore Town, a large interactive 3D city for the web, covering rendering performance, architecture, asset pipelines and production releases. Before that, at Vimeo, I built real-time 3D experiences, including a GPU-driven crowd simulation in WebGL, Three.js and TypeScript. A strong background in mathematics and physics has helped me throughout my career, and I enjoy problems where they meet real-time graphics and GPU programming.

ENEMINDS · 2016

Co-founder, lead developer

Redungeon (Unity / C#), published by Nitrome: Apple Editors' Choice, 1M+ downloads per platform.

VIMEO · 2022–2025

3D lead, Vimeo Venues

Crowd engine for virtual events: up to 10,000 avatars at a stable 60+ FPS on everyday laptops. WebGL, Three.js and TypeScript. I prototyped on the CPU, then moved the sim to the GPU.

VIMEO · LIVE EVENTS

Frontend at scale

React / TypeScript on a large production platform. UI performance, a shared design system, and tests that meant fewer regressions and more frequent releases.

REDCORE · 2025–2026

Product owner & lead engineer

RedCore Town was in rough shape. I stabilised it, cut scene loading 3x, and shipped a standalone Angular app as an npm package. Then I owned architecture, assets and releases.

02 · FIT

What InfraCloud3D needs, what I bring

YOU NEED	STATUS	EVIDENCE
Large, complex 3D scenes at interactive frame rates	SHIPPED	Vimeo Venues: up to 10,000 avatars at 60+ FPS in the browser · RedCore Town: scene loading cut 3x (GLB, Draco, memory, S3 delivery)
WebGPU pipelines & compute	BUILT	webgpu-crowd: WGSL compute, spatial hash grid, 1k to 100k agents · webgpu-ocean: TSL compute FFT
Angular + TypeScript frontend	SHIPPED	Standalone Angular app shipped as an npm package (RedCore) · React / TS at Vimeo scale
Data-driven / geospatial 3D	BUILT	Globe Dashboard: live USGS feed, instanced markers, synced virtualised UI
Guiding other developers	PROVEN	“Three.js from Scratch”, 6 parts incl. WebGPU & TSL and post-processing
LuciadRIA · point clouds · Gaussian Splatting	TRANSFER	Same GPU fundamentals: buffers, instancing, LOD, streaming, compute. Ramp-up plan on slide 11

OWNERSHIP

I moved from stabilising Town to end-to-end technical responsibility.

DEPTH

I have built GPU-driven real-time 3D, not only UI components.

JUDGMENT

I balance an ambitious design with what a real release can hold.

03 · BIG SCENES IN PRODUCTION

Frame budgets I've held for real users

VIMEO · VENUES

10,000 avatars at a stable 60+ FPS

- I built the crowd engine and owned simulation, rendering and product integration, in a team of 10.
- The target was up to 10,000 attendees, moving in real time, with physics-based behaviour.
- I built a CPU prototype first. The team liked it. The same approach would not scale, so I moved the sim to the GPU.
- WebGL fragment shaders and ping-pong buffers. Positions and velocities lived in 2D floating-point textures, so I was not constantly transferring that state back to the CPU.
- Passes: state updates, velocity limits and motion effects, then the particle interactions. A fragment shader cannot safely update two particles at once, so each pass reads a stable previous state.
- The speakers stayed interactive inside a protected zone. I put a signed distance field in the sim, so an attendee was pushed back out even if a user dragged them in.

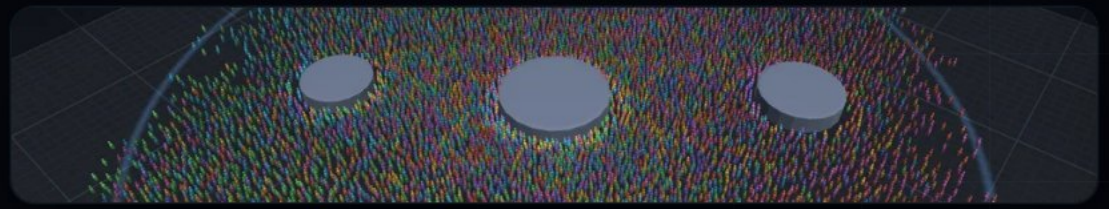
REDCORE · REDCORE TOWN

3x faster scene loading

- When I joined, Town was in rough technical shape. I stabilised it: bugs, rendering and performance. My scope then grew to the whole product, through to release.
- Interaction raycasting ran on every tick. I changed it to run only on the events that needed a result.
- I introduced Draco in the asset pipeline. Model downloads got smaller and loading improved, without a noticeable loss of visual quality.
- I reviewed expensive rendering effects and removed them, or replaced them with cheaper ones.
- I started with measurement rather than guessing. Chrome DevTools, frame time, CPU and GPU, memory and loading. One change at a time, then I compared the same scenes.
- Scene loading cut 3x: GLB assets, memory and S3 delivery. Models and textures were no longer bundled into the app package.

Compute shaders doing the heavy lifting

The crowd is raw WebGPU. The ocean is Three.js. In both, the simulation stays on the GPU.



webgpu-crowd · 1k → 100k agents

- Raw WebGPU and WGSL, no engine. Positions, velocities and goals never leave the GPU, so there is no per-agent CPU work each frame.
- Buffers are allocated once for 100,000 agents. Changing the count only changes the dispatch and the instance count. Default is 10,000. The panel runs from 1,000 to 100,000.
- Ping-pong vec4 state: xz position and xz velocity. A second buffer holds the goal, a speed scale, and which of six gates the agent is walking toward.
- Spatial hash: an 80×80 grid of 2×2 cells. One compute pass, five dispatches, workgroups of 256. Clear, atomic bin, a single-workgroup Hillis–Steele scan, scatter, then steer.
- Steering reads the 3×3 neighbourhood. The cell is at least the neighbour radius, and each cell is capped at 32 agents. The write goes into the other ping-pong buffer.
- One instanced draw. The vertex shader reads the latest buffer with instance_index. About 150 triangles per agent, 4× MSAA. At 100,000 the mesh is the limit, not the simulation.

`iodiot.github.io/webgpu-crowd`



webgpu-ocean · TSL compute FFT

- Three.js WebGPURenderer. The spectrum, the FFT and the water shader are TSL nodes, compiled to WGSL. The simulation is not read back to the CPU.
- Three cascades, each 256×256. Patch sizes are 250 m, 37 m and 7.3 m, covering swell down to ripples of about 6 cm. The sizes are not integer multiples, so the tiling does not line up.
- JONSWAP or Phillips, with deep-water dispersion $\omega = \sqrt{(gk)}$. h0 is rebuilt on the GPU only when the spectrum changes. Breeze is about 2 m significant wave height.
- One compute pass per frame, 12 dispatches. Each cascade evolves, then a row IFFT, a column IFFT, then assemble.
- The IFFT is a radix-2 Stockham in workgroup memory. One workgroup of 128 threads owns a row or a column, eight stages. Eight real fields are packed into four complex transforms.
- Assemble writes two rgba16float textures: displacement plus foam, and slopes plus the Jacobian. Foam is where the surface folds, then it decays. Choppiness is the horizontal displacement.
- The surface is one camera-centred polar grid: about 190,000 vertices out to 9 km, about 460,000 triangles. A MeshBasicNodeMaterial displaces the vertices and rebuilds the normal from the slopes.

`iodiot.github.io/webgpu-ocean`

Globe Dashboard

A live earthquake dashboard around a 3D globe, streaming the USGS real-time feed.

- Every event is one instanced marker; recent events pulse in a shader
- Hover or click to fly the camera to an event
- Virtualised list, filters, KPI cards and SVG charts stay in sync with the globe
- Timeline playback, shareable URL state, keyboard navigation, snapshot fallback when the feed is down

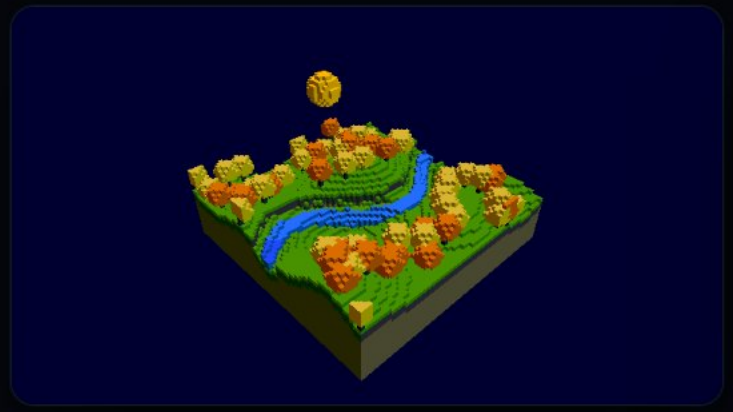
React · TypeScript · Three.js / R3F · zustand · Vitest

iodiot.github.io/globe-dashboard



No engine, just shaders: voxels, rays, fluids

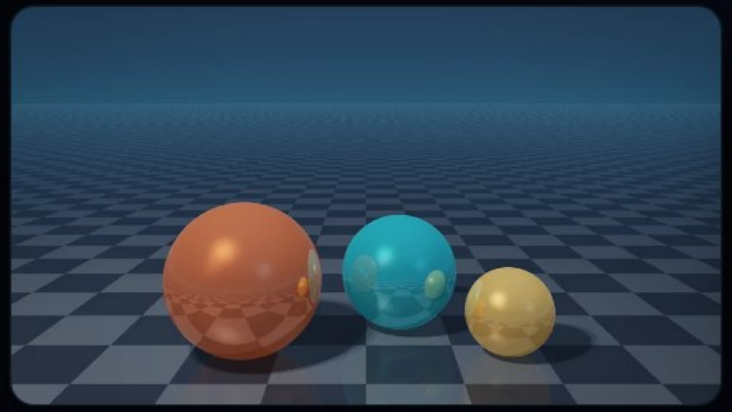
No engine in between. Walking a voxel grid in a 3D texture is the same kind of problem as traversing dense spatial data. The shader raycasting and signed distance fields I was experimenting with here are what I later used inside a physics sim, not only to draw a shape.



GpuVoxels

Ray casting a 64×64×64 voxel grid stored as an OpenGL 3D texture; every pixel walks the grid block by block until it hits.

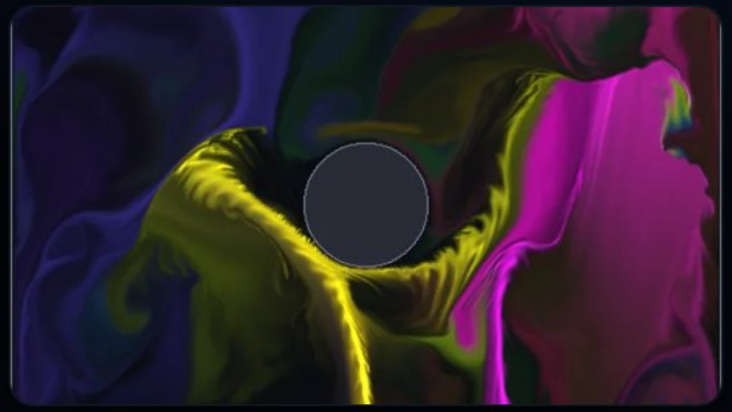
github.com/iodiot/GpuVoxels



GpuRayTracer

Live ray tracing in a single shader: two light bounces between spheres, soft shadows from an area light.

github.com/iodiot/GpuRayTracer



GpuFluid

Navier–Stokes on the GPU: advection, buoyancy, Jacobi pressure solve as shader passes; vorticity confinement, 512 grid.

github.com/iodiot/GpuFluid

C++ · OpenGL · GLSL · 2014–2015 · refreshed 2026 · CMake + GLFW

“Three.js from Scratch”: a free series for frontend devs

Six parts so far, from a first lit cube to shaders, WebGPU and post-processing. Diagrams, real screenshots and a live demo in every part. If I can walk a frontend developer through the migration, I can help a team do it the same way.



Part 5 · WebGPU and TSL

Switch to WebGPURenderer, rebuild materials as TSL node graphs, read the WGSL it generates, then 40,000 particles simulated entirely on the GPU. WebGL 2 fallback.



Part 6 · Post-Processing and Bloom

Render targets and ping-pong by hand, EffectComposer, a custom lens shader, then the same chain on WebGPU with TSL’s RenderPipeline, and selective bloom. Ping-pong means one buffer holds the previous state, the next pass writes the other, then they swap.

iodiot.github.io/3d-tutorial

08 · PRODUCT ENGINEERING

The app around the 3D: Angular + TypeScript

REDCORE · 2025–2026

- I joined Town in rough shape. First job: stabilise bugs, rendering and performance.
- Scope then grew: new 3D content, seasonal experiences, interactive features, and the production releases.
- I split Town out of the larger RedCore app. Standalone Angular application, existing 3D migrated, npm package the main platform consumes.
- Models and textures moved to S3, out of that package. The package stayed smaller, and assets shipped separately from app releases.
- I worked with DevOps on the pipeline, and I built a drag-and-drop tool so designers could upload and test models without a developer in the loop.
- Town 2100 was more ambitious than the deadline. Rebuilding from scratch needed more time and assets than we had. With design I kept the main look, the core ideas and the mini-games, and we simplified the expensive parts. Management approved.

VIMEO · 2022–2025

- Live Events features on a large production platform, in React and TypeScript.
- I focused on UI performance and contributed to the shared design system.
- Better automated testing: fewer regressions, and we could release more often.
- 3D lead for Venues, inside a team of 10. WebGL, Three.js and TypeScript.
- The crowd was the deep technical problem: a CPU prototype, then GPU fragment shaders and ping-pong buffers.

My title was Senior Engineer. I was already doing the broader job: architecture, cross-team work with design and DevOps, and a scope we could actually ship.

09 · HONEST GAPS, FAST RAMP-UP

New to me, built on what I already do

I have not shipped LuciadRIA, point clouds or Gaussian Splatting in production. These are problems I already solve in other shapes, and I ramp up on the API and on real data.

LuciadRIA

Another engine on the same fundamentals: scene graph, layers, camera, GPU buffers. I'd learn it through the API docs, your codebase and a small spike on real data.

Point clouds & meshes

Huge instanced draws, spatial hashing, GPU memory and asset-delivery budgets are what I've shipped. Point-cloud LOD and streaming is that same discipline at a larger scale.

Gaussian Splatting

Sorting and blending millions of splats is a GPU compute and rendering problem: the WebGPU compute and custom-shader work I do today.

IF WE SAY POINT CLOUD

- A point cloud is 3D samples, often from LiDAR or photogrammetry. XYZ, sometimes colour.
- LOD: a simpler representation for what is far away. Streaming: only the chunks that fit in RAM and GPU memory.
- An octree divides space into eight, for queries and LOD. A BVH groups geometry under bounding volumes to speed ray tests. Frustum culling skips whatever is outside the camera.

IF WE SAY GAUSSIAN SPLATTING

- Many semi-transparent 3D Gaussians. Projected and overlapped, they read as a continuous image.
- Each one stores position, scale, rotation, opacity and colour. Spherical harmonics cover view-dependent colour.
- Project to screen-space ellipses, cull, sort by depth, alpha-blend. Wrong order shows up as blend artefacts, and the sort itself is a performance problem.
- Usually optimised from photos with camera poses. Initialising Gaussians from points is not that trained reconstruction.
- Large captures still need streaming, visibility selection and a GPU memory budget. Gaussian Splatting and geometry-based measurement solve different needs.

ALSO ON MY RADAR Large coordinate spaces: float precision, relative-to-eye / floating origin, depth precision on city- and bridge-scale scenes.

How I'd approach the first 90 days

DAYS 1–30

Learn & measure

- LuciadRIA deep dive on your codebase
- Profile real datasets on target hardware
- Chrome DevTools: frame time, CPU and GPU, memory, load time
- Set budgets: frame time, GPU memory, load time
- Find the bottleneck first. Change one thing at a time, then compare

DAYS 31–60

Fix the biggest bottleneck

- LOD and streaming for meshes and point clouds, inside RAM and GPU-memory budgets
- Frustum culling, instancing, worker offloading
- Automated perf checks so wins don't regress

DAYS 61–90

WebGPU migration path

- Feature detection with a WebGL fallback
- First compute workload moved to the GPU
- Spike: Gaussian Splatting on a real survey. Project, sort, blend, and watch streaming and GPU memory

11 · QUESTIONS FOR YOU

What I'd like to understand

1 Where does LuciadRIA hit its limits today, and where would WebGPU come in?

I want the real frame-time and GPU-memory wall, and which workload should move to compute first.

2 Typical dataset size (points, triangles, GB) and target hardware?

That decides LOD, streaming, and what actually fits in RAM and on the GPU.

3 Team size, and how 3D work is split between frontend and backend?

I have split this before: the app as an npm package, assets on S3, designers uploading models themselves.

THANK YOU

Let’s build it on the GPU.

Mike Smahin · Senior Frontend / 3D Engineer · Barcelona

I enjoy complex graphics problems where mathematics, physics and software architecture meet. I have already owned the delivery of a 3D product end to end, and I want to apply that to spatial-data visualisation.

Email	iodiot@gmail.com
Portfolio	iodiot.github.io
GitHub	github.com/iodiot
X	@iodiot
Tutorial	iodiot.github.io/3d-tutorial
Crowd	iodiot.github.io/webgpu-crowd
Ocean	iodiot.github.io/webgpu-ocean
Globe	iodiot.github.io/globe-dashboard